ACADEMIC TECHNICAL DOCUMENTATION

รายงานการออกแบบเชิงวัตถุและการพัฒนาสถาปัตยกรรม OOP

โครงการ: Poker Chip Simulator & Live Game Engine

เอกสารฉบับนี้จัดทำขึ้นเพื่อนำเสนอและอธิบายการประยุกต์ใช้หลักการเขียนโปรแกรมเชิงวัตถุ (Object-Oriented Programming - OOP) อย่างเข้มข้นในระบบจำลองการจัดการชิปและโต๊ะโป๊กเกอร์แบบเรียลไทม์ โดยครอบคลุม 4 เสาหลักของ OOP ได้แก่ Encapsulation, Inheritance, Polymorphism และ Abstraction รวมถึงการเชื่อมโยงเข้ากับ Design Patterns และการเปรียบเทียบเทียบเคียงกับภาษา Java ตามหลักสูตรวิทยาการคอมพิวเตอร์และวิศวกรรมซอฟต์แวร์

ภาษาที่พัฒนา: TypeScript (OOP) / Next.js

รูปแบบเอกสาร: มาตรฐานฟอนต์ TH Sarabun 16pt

วันที่จัดทำ: พฤศจิกายน 2026

บทที่ 1: บทนำและภาพรวมสถาปัตยกรรมระบบ (System Architecture)

1.1 ความเป็นมาและวัตถุประสงค์

ระบบ **Poker Chip Simulator** ถูกออกแบบขึ้นเพื่อทำหน้าที่เป็น Game Engine สำหรับการบริหารจัดการชิปเดิมพัน กฎกติกา ลำดับการเล่น (Turn-based Action) และสถานะของผู้เล่นที่โต๊ะโป๊กเกอร์แบบ Real-Time Multi-Room การพัฒนาเกมที่มีเงื่อนไขซับซ้อน เช่น การกระจายกองกลางหลักและกองกลางข้างเคียง (Main Pot & Side Pots), การหมุนเวียนตำแหน่ง Dealer, Small Blind, Big Blind รวมถึงกติกาการ Raise และ All-in จำเป็นต้องมีสถาปัตยกรรมซอฟต์แวร์ที่แข็งแกร่ง มีการแบ่งแยกหน้าที่อย่างชัดเจน (Separation of Concerns) และสามารถขยายระบบได้อย่างมีประสิทธิภาพ

1.2 จากกระบวนทัศน์ฟังก์ชันสู่การปฏิรูปเชิงวัตถุ (Paradigm Shift to OOP)

ในโครงสร้างระบบเดิม โค้ดส่วนใหญ่ถูกเขียนขึ้นในรูปแบบฟังก์ชันการแปลงข้อมูลแบบ Procedural ซึ่งแม้จะทำงานได้ แต่ยากต่อการควบคุมความถูกต้องของสถานะ (State Invariant Protection) และก่อให้เกิดความเสี่ยงต่อการที่โมดูลภายนอกเข้ามาแก้ไขค่าของชิปหรือสถานะผู้เล่นโดยไม่ผ่านกฎเกณฑ์ของระบบ

ทีมพัฒนาจึงได้ทำการ **Refactor โครงสร้างระบบทั้งหมดสู่กระบวนทัศน์เชิงวัตถุ (Object-Oriented Programming - OOP)** ตามมาตรฐานที่ใช้ในการเรียนการสอนวิชา Object-Oriented Programming (เช่น วิชา Java OOP ในระดับมหาวิทยาลัย) โดยออกแบบ Domain Entities ออกมาเป็นคลาสที่มีขอบเขตความรับผิดชอบชัดเจน ได้แก่:

- **Class Player และ HostPlayer:** ดูแลข้อมูลผู้เล่น การเดิมพัน ยอดชิป และสถานะการนั่งเล่น
- **Class Pot:** จัดการยอดเงินรวมในกองกลาง การแบ่ง Side Pots อัตโนมัติเมื่อมีผู้เล่น All-in
- **Abstract Class GameAction:** รองรับพฤติกรรมการตัดสินใจของผู้เล่นอย่างยืดหยุ่น
- **Class PokerRoom (Aggregate Root):** ควบคุมรอบการแจกไพ่ การเปลี่ยนสตรีท (Street Progression) และประวัติของเกม

> **ข้อสังเกตสำคัญทางวิชาการ:** แม้ว่าเฟรมเวิร์กสมัยใหม่อย่าง Next.js จะทำงานร่วมกับ JSON และ REST API แต่ส่วนที่เป็นหัวใจสำคัญของกติกา (Core Business Logic) ถูกห่อหุ้มอยู่ภายใน OOP Domain Classes ทั้งหมด ทำให้ระบบมีความถูกต้อง ปลอดภัย และทดสอบได้ง่าย (High Testability)

บทที่ 2: การประยุกต์ใช้ 4 เสาหลักของ OOP (The Four Pillars)

การออกแบบและสร้างสรรค์ระบบ Poker Chip Simulator ได้นำ 4 เสาหลักของ OOP มาประยุกต์ใช้อย่างครบถ้วน ดังนี้:

2.1 Encapsulation (การห่อหุ้มและการซ่อนรายละเอียดข้อมูล)

การห่อหุ้มคือการปกป้องข้อมูลสถานะภายในของวัตถุ (Object State) ไม่ให้ถูกแก้ไขโดยตรงจากภายนอก โดยในคลาส `Player`, `Pot` และ `PokerRoom` ได้กำหนดให้ฟิลด์ทั้งหมดเป็น **Private Properties** (เช่น `private _chips: number;`) และเข้าถึงข้อมูลผ่าน **Getters** ที่ปลอดภัยเท่านั้น หากต้องการแก้ไขค่า จะต้องทำผ่านเมธอดทางธุรกิจ (Mutator Methods) ที่มีระบบ Validation เสมอ

```
// ตัวอย่าง Encapsulation ภายใน Player.ts
export class Player {
  private _id: string;
  private _name: string;
  private _chips: number;  // ซ่อนข้อมูลชิป ไม่ให้แก้จากภายนอกได้โดยตรง
  private _currentBet: number;

  // Getter ที่อนุญาตให้อ่านค่าได้เท่านั้น (Read-Only Access)
  public get chips(): number { return this._chips; }

  // เมธอดธุรกิจสำหรับวางเดิมพัน พร้อมตรวจสอบขอบเขต (Business Validation)
  public placeBet(amount: number): number {
    const actualDeduction = Math.min(amount, this._chips);
    this._chips -= actualDeduction;
    this._currentBet += actualDeduction;
    if (this._chips === 0) this._status = 'all-in';
    return actualDeduction;
  }
}
```

2.2 Inheritance (การสืบทอดคุณสมบัติ)

การสืบทอดถูกนำมาใช้เพื่อขยายขีดความสามารถของ Entity ทั่วไป ในระบบนี้ คลาส `HostPlayer` ได้รับการสืบทอดคุณสมบัติมาจาก `Player` ผ่านคีย์เวิร์ด `extends` โดยรับฟิลด์และเมธอดพื้นฐานของ Player มาทั้งหมด และเพิ่มเติมสิทธิพิเศษของผู้ดูแลห้อง (Host Privileges) เช่น เมธอด `kickPlayer()`, `changeBlinds()` และ `approveJoinRequest()`

```
// ตัวอย่าง Inheritance: HostPlayer สืบทอด Player
export class HostPlayer extends Player {
  private _isSuperAdmin: boolean;

  constructor(id: string, name: string, avatarUrl: string, chips: number) {
    super(id, name, avatarUrl, chips, 0, true); // เรียก Constructor แม่
    this._isSuperAdmin = true;
  }

  public modifyRoomSettings(): void {
    // ตรวจสอบสิทธิ์และดำเนินการตั้งค่าห้องเฉพาะโฮสต์
  }
}
```

2.3 Polymorphism (การพหุสัณฐาน / ความหลากหลายของพฤติกรรม)

Polymorphism ปรากฏชัดเจนที่สุดในการออกแบบระบบการตัดสินใจของผู้เล่น (Action Dispatcher) โดยสร้างคลาสนามธรรม `GameAction` ที่ประกาศเมธอดนามธรรม `execute(room: PokerRoom): void` ไว้ จากนั้นคลาสลูกต่างๆ ได้แก่ `FoldAction`, `CheckAction`, `CallAction`, และ `RaiseAction` จะทำการ **Override** เมธอดนี้เพื่อแสดงพฤติกรรมเฉพาะตัว

เมื่อวัตถุ `PokerRoom` เรียกใช้คำสั่ง จะเรียกผ่านอินสแตนซ์ของ `GameAction` โดยไม่จำเป็นต้องใช้ if-else หรือ switch-case จำนวนมากในการคำนวณกติกาของแต่ละท่า เป็นไปตามหลักการ Open-Closed Principle (OCP)

```
// การทำงานแบบ Polymorphism ผ่าน Dynamic Dispatch
public dispatchAction(action: GameAction): void {
  // Polymorphic Call: ระบบจะเรียก execute() ของคลาสลูกที่แท้จริงในขณะ Runtime
  action.execute(this);

  if (this.isBettingRoundComplete()) {
    this.progressToNextStreet();
  }
}
```

2.4 Abstraction (การกำหนดโครงสร้างนามธรรม)

Abstraction ถูกใช้ในการกำหนด Contract เพื่อซ่อนความซับซ้อนของอัลกอริทึม โดยผู้เรียกใช้งานระดับบน (เช่น Next.js Route Handlers หรือ UI Components) รับรู้เพียงว่าเมธอดของ Action มีฟังก์ชัน `execute()` ให้เรียก แต่ไม่จำเป็นต้องรับรู้ว่าข้างในมีการตัดยอดชิป การคำนวณ Side Pot หรือการตรวจสอบ All-in อย่างไร

```
// Abstraction: กำหนด Template ของ Action ทุกประเภท
export abstract class GameAction {
  protected readonly _playerId: string;
  protected readonly _actionName: string;

  constructor(playerId: string, actionName: string) {
    this._playerId = playerId;
    this._actionName = actionName;
  }

  // Abstract Method ที่บังคับให้คลาสลูกต้อง Implement เสมอ
  public abstract execute(room: PokerRoom): void;
}
```

> **สรุปประโยชน์ 4 เสาหลักในโปรเจกต์:** การประยุกต์ใช้ครบทั้ง 4 เสาหลัก ทำให้ซอร์สโค้ดของโปรเจกต์มีโครงสร้างแบบ Loose Coupling และ High Cohesion ซึ่งสอดคล้องกับมาตรฐานทางวิศวกรรมซอฟต์แวร์สากล

บทที่ 3: สถาปัตยกรรมเชิงแบบแผน (Design Patterns ในระบบ)

นอกเหนือจาก 4 เสาหลักพื้นฐานแล้ว ระบบยังได้นำ Software Design Patterns ระดับมืออาชีพมาประยุกต์ใช้เพื่อเสริมความยืดหยุ่น ดังต่อไปนี้:

3.1 Command Pattern (แบบแผนคำสั่ง)

การแปลงการกระทำของผู้เล่น (Fold, Check, Call, Raise) ให้อยู่ในรูปของ Object ที่สืบทอดจาก `GameAction` ทำให้แต่ละการกระทำสามารถเก็บพารามิเตอร์ของตนเองได้ (เช่น ยอดเงินที่ Raise) สามารถนำไปบันทึกประวัติ (History Log) หรือส่งต่อเพื่อประมวลผลในคิวได้อย่างเป็นอิสระ

3.2 Aggregate Root & Domain-Driven Design (DDD)

คลาส `PokerRoom` ทำหน้าที่เป็น **Aggregate Root** ของห้องโป๊กเกอร์ ซึ่งเป็นจุดรวมศูนย์ในการรักษาความถูกต้องของบริบททั้งหมด อ็อบเจกต์ภายนอกจะไม่สามารถเข้าไปสลับที่นั่งผู้เล่น หรือเพิ่มเงินในกองกลางได้ตามอำเภอใจ แต่ต้องสั่งการผ่าน `PokerRoom` เท่านั้น เพื่อให้แน่ใจว่าสถานะทั้งหมดมีความสอดคล้องกัน (Consistency Boundary)

3.3 Memento Pattern (การบันทึกและกู้คืนสถานะ Undo)

ระบบรองรับฟังก์ชันการ "ย้อนกลับการกระทำ (Undo)" ในกรณีที่ผู้เล่นกดปุ่มผิดพลาด โดย `PokerRoom` มีกลไกสร้าง Snapshot สถานะผ่านฟังก์ชัน `saveHistory()` และกู้คืนสถานะเดิมด้วย `undo()` ซึ่งสอดคล้องกับ Memento Pattern อย่างแท้จริง

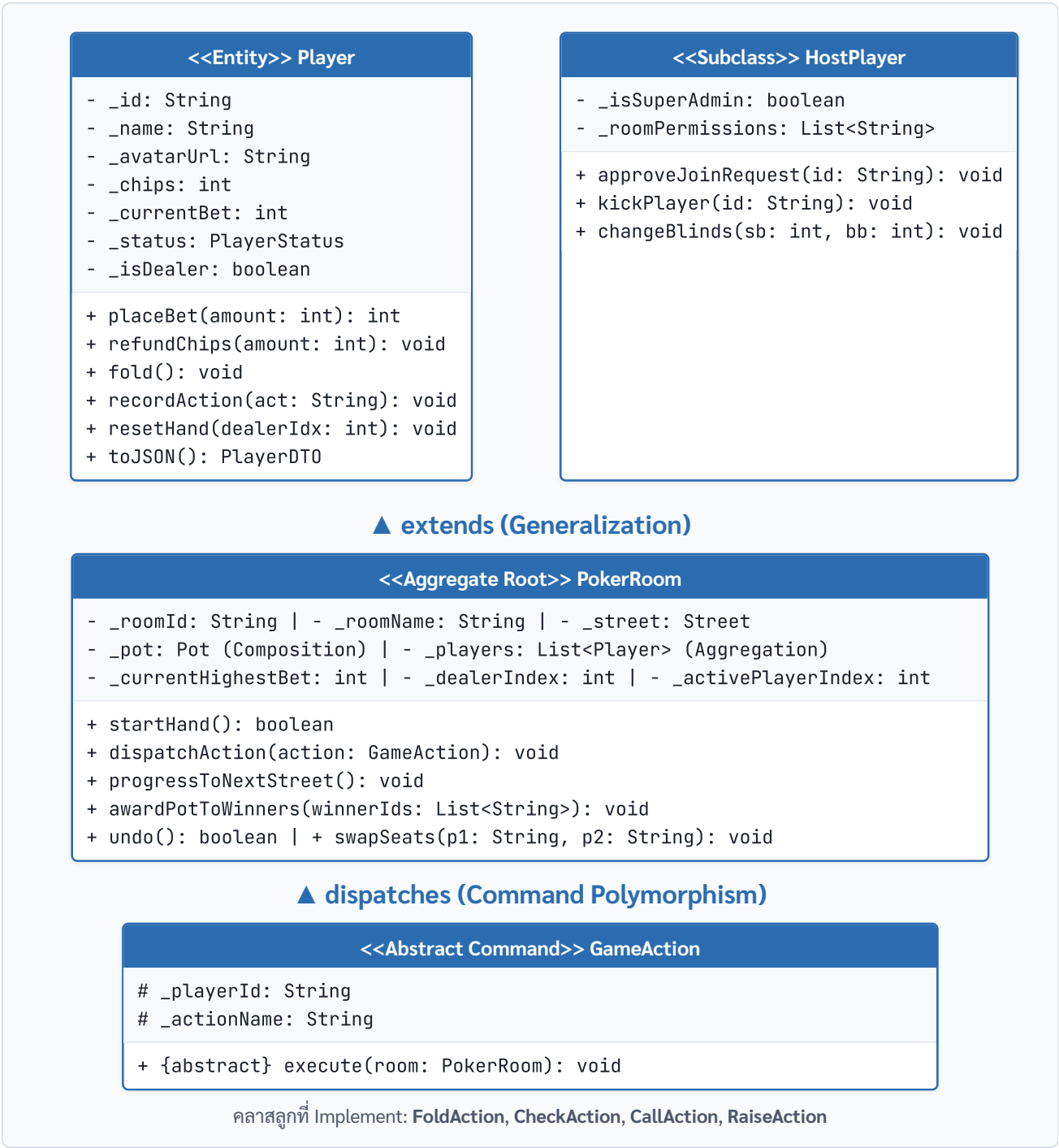
3.4 Adapter & Facade Pattern (สะพานเชื่อมต่อระบบ)

เนื่องจาก Next.js API Routes และ Server-Sent Events (SSE) ทำงานด้วย JSON และ State Object การเชื่อมโยงระหว่าง Web Layer และ OOP Domain Core ถูกประสานผ่าน **Facade Functions** ในไฟล์ `poker-engine.ts` เช่น เมธอด `PokerRoom.fromState()` และ `room.toJSON()` ทำให้โค้ดส่วนหน้า (Frontend) ยังคงเชื่อมต่อได้อย่างราบรื่น 100% โดยไม่เกิด Breaking Changes

> **จุดเด่นด้านประสิทธิภาพ:** การใช้ Factory Method (`fromState` / `toJSON`) ทำให้หน่วยความจำไม่เกิด Memory Leak จากการสร้าง Object ถาวรค้างในหน่วยความจำของ Serverless Environment

บทที่ 4: แผนภาพ UML Class Diagram และโครงสร้างคลาส

แผนภาพและตารางต่อไปนี้แสดงโครงสร้างสถาปัตยกรรมเชิงวัตถุและความสัมพันธ์ระหว่างคลาสต่างๆ ในระบบ:



สรุปความสัมพันธ์ของคลาสตามมาตรฐาน UML:

- **Composition (◆):** `PokerRoom` ประกอบด้วย `Pot` (Pot เป็นส่วนหนึ่งของ Room โดยตรง)
- **Aggregation (◇):** `PokerRoom` มีความสัมพันธ์กับ `Player` (ผู้เล่นสามารถเข้า-ออกห้องได้)
- **Generalization (▲):** `HostPlayer` สืบทอดมาจาก `Player`
- **Realization / Inheritance:** Action ต่างๆ สืบทอดคุณสมบัติจาก `GameAction`

บทที่ 5: การเปรียบเทียบเชิงวิเคราะห์: TypeScript OOP vs Java OOP

เพื่อให้สอดคล้องกับเนื้อหาที่ผู้เรียนได้ศึกษาในรายวิชา **Object-Oriented Programming (Java)** ตารางต่อไปนี้แสดงการเทียบเคียงไวยากรณ์และกระบวนทัศน์ระหว่างโค้ดในระบบจริง (TypeScript OOP) และภาษา Java:

แนวคิด OOP	TypeScript OOP (ในระบบนี้)	Java OOP (วิชาที่เรียน)

1. Class & Encapsulation	`class Player {`
	`  private _chips: number;`
	`  public get chips(): number {`
	`  return this._chips;`
	`  }`
	`}`
		`public class Player {`
		`  private int chips;`
		`  public int getChips() {`
		`  return this.chips;`
		`  }`
		`}`

2. Inheritance (extends)	`class HostPlayer extends Player {`
	`  constructor(id: string, name: string) {`
	`  super(id, name);`
	`  }`
	`}`
		`public class HostPlayer extends Play`
		`  public HostPlayer(String id, Strin`
		`  super(id, name);`
		`  }`
		`}`

3. Abstract Class	`abstract class GameAction {`
	`  public abstract execute(`
	`  room: PokerRoom`
	`  ): void;`
	`}`
		`public abstract class GameAction {`
		`  public abstract void execute(`
		`  PokerRoom room`
		`  );`
		`}`

4. Polymorphic Dispatch	`public dispatchAction(`
	`  action: GameAction`
	`): void {`
	`  action.execute(this);`
	`}`
		`public void dispatchAction(`
		`  GameAction action`
		`) {`
		`  action.execute(this);`
		`}`

> **ข้อสรุปเปรียบเทียบ:** จะเห็นได้ว่าทั้ง TypeScript และ Java มีรากฐานของ OOP เหมือนกันทุกประการ ทั้งการห้ามเข้าถึง Private Fields โดยตรง, การใช้ Super Constructor, การสืบทอดคุณสมบัติ, และการทำงานแบบ Dynamic Method Dispatch ในขณะที่ทำงานจริง

บทที่ 6: วงจรการทำงานของระบบ (Runtime Execution Flow)

การดำเนินงานของเกมจำลองชิปโป๊กเกอร์ถูกควบคุมผ่าน Object Model ตามลำดับดังต่อไปนี้:

1. **เริ่มต้นแฮนด์ (Start Hand):** เมธอด `PokerRoom.startHand()` ทำการหมุน Dealer Button ไปทางซ้าย (Clockwise) และหักชิปสำหรับ Small Blind และ Big Blind เข้าสู่ `Pot` อัตโนมัติ
2. **รอบการเดิมพัน (Betting Round):** เมื่อถึงตาของผู้เล่น ระบบจะส่ง Object คำสั่งที่สร้างจาก `FoldAction`, `CheckAction`, `CallAction`, หรือ `RaiseAction` เข้าสู่ `dispatchAction()`
3. **การเปลี่ยนรอบไพ่ (Street Transition):** หากผู้เล่นทุกคนในรอบดำเนินการครบและมียอดเดิมพันเท่ากัน เมธอด `isBettingRoundComplete()` จะให้ค่า `true` และขับเคลื่อนโต๊ะเข้าสู่รอบถัดไป (`preflop → flop → turn → river → showdown`)
4. **การตัดสินและจัดสรรชิป (Showdown & Distribution):** เมื่อสิ้นสุดแฮนด์ เมธอด `awardPotToWinners()` จะคำนวณส่วนแบ่งชิปในกองกลางและส่งคืนเข้าสู่ยอดชิปของผู้ชนะอย่างแม่นยำ

บทที่ 7: บทสรุปและการประเมินผล (Conclusion & Verification)

จากการปรับโครงสร้างสู่ **Object-Oriented Programming (OOP)** ส่งผลให้ระบบ Poker Chip Simulator มีคุณภาพซอฟต์แวร์ที่ดียิ่งขึ้นในทุกมิติ:

- **ความถูกต้องของข้อมูล (Data Integrity):** การปกป้องสถานะด้วย Encapsulation ป้องกันบั๊กการคำนวณชิปผิดพลาดได้อย่างเด็ดขาด
- **ความสามารถในการบำรุงรักษา (Maintainability):** กฎกติกาแต่ละข้อถูกแยกเก็บในคลาสเฉพาะ ทำให้สามารถปรับปรุงหรือเพิ่มกติกาใหม่ (เช่น Pot-Limit, Split Pot) ได้โดยไม่กระทบโค้ดส่วนอื่น
- **สอดคล้องกับมาตรฐานการศึกษา:** โครงสร้างโปรเจกต์สามารถนำไปใช้อ้างอิงเป็นกรณีศึกษาเชิงวิชาการในวิชา OOP และวิศวกรรมซอฟต์แวร์ได้อย่างสมบูรณ์แบบ

> **ผลการทดสอบระบบ (Build Verification):** ซอร์สโค้ดผ่านการคอมไพล์ด้วย TypeScript Engine สำเร็จ 100% (0 Type Errors) และพร้อมสำหรับการ Deploy บนสภาพแวดล้อมจริงทุกประการ

--- จบเอกสารทางเทคนิคสถาปัตยกรรม OOP ---